

MSc Project 1: Time-Series-Enhanced Predictive Process Monitoring

Phase Review 2

Michał Durkalec, Konstantinos Sakkas, and Roman Ležovič

Supervised Process Intelligence (SPI) Lab,
RWTH Aachen University, Aachen, Germany
`office@pads.rwth-aachen.de`

Abstract. This document covers Sprint 2 of the project, reporting on its objectives, technical implementation, and retrospective. The main achievements were: the derivation and integration of active-case-count time-series features from the BPI Challenge 2017 event log, aligned with case prefixes using timestamps; the implementation of prediction target functions for remaining time and loan outcome; and the introduction of a parametric pipeline execution system with a CLI entry point. A classification evaluation module (accuracy, F1) was also added alongside the existing regression metrics. The sprint concludes with all pipeline components wired end-to-end and runnable from a single command, setting the stage for the full comparative evaluation and final report.

Keywords: Predictive Process Monitoring · Time-Series Features · Process Mining · Feature Engineering · Parametric Pipeline

1 Introduction & Objectives

1.1 Sprint Goals

Sprint 2 built directly on the modular pipeline delivered in Sprint 1 by pursuing three parallel tracks. The first was *time-series incorporation*: deriving and aligning external temporal signals with the case prefixes generated in Sprint 1. The second was *model extension*: training and evaluating hybrid models that combine the existing log-based feature set with the newly derived time-series features, and comparing their performance against the Sprint 1 baselines. The third was *infrastructure improvement*: introducing parametric run configuration, a CLI entry point, and a development dataset to reduce the iteration cycle during experimentation.

1.2 Team Organization

Role assignments were rotated as planned at the end of Sprint 1 (Table 2). Konstantinos took over as Project Coordinator and Scrum Master. Michał moved to

Technical Lead, focusing on infrastructure and pipeline parameterization. Roman optimized the pipeline infrastructure, enabling the previously unusable onehot encoded baseline features. Konstantinos also retained the Data and Preprocessing Lead role, driving time-series feature extraction and alignment. Issue tracking and sprint management continued exclusively within GitHub Issues following the consolidation from Trello in Sprint 1.

2 Technical Implementation

Sprint 2 extended the existing pipeline with time-series functionality, prediction targets, and infrastructure improvements. The modular strategy-pattern architecture introduced in Sprint 1 absorbed these additions without structural changes to existing code. The following subsections describe the time-series pipeline, the target functions and extended evaluation, and the infrastructure improvements delivered across the closed issues.

2.1 Time-Series Feature Pipeline

Time-series signals were derived directly from the BPI 2017 event log [1] rather than from an external source, treating the log itself as a source of workload dynamics. The derivation and integration proceeded in three stages, all implemented in `features/time_series_features.py`.

Preprocessing (Issue #54). Case start and end times are extracted from the cleaned event log and used to build a continuous hourly timestamp grid spanning the full log. For each hour, the number of concurrently active cases is computed by counting cases whose lifetime interval contains that timestamp. The result is a regularly spaced active-case-count time series requiring no missing-value imputation.

Feature extraction (Issue #55). Rolling-window statistics—mean, maximum, and standard deviation—are computed over fixed windows of 1 h, 6 h, 12 h, and 24 h for the active-case-count series. Lag features (the raw count offset by 1, 6, 12, and 24 steps) and their first-differences (trend) are appended, together with temporal indicators (hour of day, day of week, and a weekend flag). The result is a fixed-length feature vector representing the workload context observable at the time a given prefix ends.

Alignment (Issue #56). Time-series feature vectors are joined to case prefixes using the timestamp of the last event in each prefix as the alignment key. Temporal mismatches are resolved by a backward-looking lookup via `merge_asof`: each prefix receives the most recently available time-series observation at or before its cutoff timestamp.

The global time-series deliverable (Issue #52) integrates these three stages into the existing pipeline as an `ActiveCaseCountFeature` class implementing

the `PrefixFeature` protocol. Multiple `PrefixFeature` instances—log-based and time-series—are composed by `extract_features_builder`, keeping the interface consistent with the log-based features from Sprint 1.

2.2 Target Functions and Extended Evaluation

Issue #71 introduced the prediction targets and extended the evaluation module to support both task types.

Target functions. `features/targets.py` provides two target generators used by the pipeline. `remaining_time_target` computes the number of hours between the last event of a prefix and the final event of the trace. `outcome_target` determines the final loan outcome—*pending*, *denied*, or *cancelled*—by inspecting which outcome-defining activity (`A_Pending`, `A_Denied`, or `A_Cancelled`) appears in the complete trace; cases with no matching activity are filtered out during preprocessing.

Classification metrics. `evaluation/metrics.py` was extended to dispatch on task type: regression predictions are still evaluated with MAE, RMSE, and R^2 ; classification predictions are now evaluated with accuracy, macro-F1, and weighted-F1, all reported per prefix length.

Extended configurations. Hybrid pipelines combining `BasicControlFlowFeatures` and `ActiveCaseCountFeature` are specified in `configs/classification_with_active_cases.yaml` and `configs/regression_with_active_cases.yaml`. Both reuse the three-algorithm design from Sprint 1 — Logistic Regression / Ridge as the linear baseline, Random Forest, and HistGradientBoosting — applied to the augmented feature set. Hyperparameter search uses `RandomizedSearchCV` with k -fold cross-validation; temporal ordering is enforced at the train/test split level via a strict time-based cutoff.

2.3 Infrastructure Improvements

Three infrastructure issues were resolved in Sprint 2. Issue #63 introduced a parametric run configuration system backed by Pydantic: a YAML configuration file controls which prediction task, feature set, model family, and hyperparameter search budget are used in a given experiment, eliminating the need to modify source code between runs. Issue #64 added a CLI entry point (`main.py`) that executes the full pipeline end-to-end from raw data to metrics given a configuration file, fulfilling REQ-IT-03 and simplifying reproduction. It also supports checkpoint save/restore and dot-notation config overrides. The development dataset is not a separate file but a `dev_mode` flag in the configuration: when enabled, `filter_dev_cases()` in `preprocessing/preprocess.py` retains only the earliest 10% of cases (by start time), reducing pipeline execution time during development. Additionally, Issue #62 optimised the one-hot encoder introduced

in Sprint 1, reducing memory consumption for high-cardinality activity-name columns and achieving an extraction speed up of around 10x.

Table 1 lists the Sprint 2 implementation steps in the order they were delivered.

Table 1. Sprint 2 implementation milestones.

Issue	Description	Key modules
#52	Global time-series data implementation (deliverable)	<code>features/time_series_features.py</code> , <code>features/extraction.py</code>
#53	Implement parametric pipeline execution (deliverable)	<code>pipeline/builder.py</code> , <code>main.py</code>
#54	Preprocess data as time-series	<code>features/time_series_features.py</code>
#55	Extract time-series features	<code>features/time_series_features.py</code>
#56	Align time-series signals with case prefixes	<code>features/time_series_features.py</code>
#58	Extended models	<code>configs</code>
#62	Optimise one-hot encoder	<code>features/log_based_features.py</code>
#63	Implement parametric run configuration	<code>config/schema.py</code> , <code>config/loader.py</code>
#64	Implement pipeline CLI entry point	<code>main.py</code>
#71	Implement target functions and classification metrics	<code>features/targets.py</code> , <code>evaluation/metrics.py</code>

3 Review & Retrospective

3.1 Successes

All planned Sprint 2 deliverables were completed. The three major deliverables—global time-series data integration (#52), target functions and extended evaluation (#71), and parametric pipeline execution (#53)—were merged and verified. A full list of closed issues is given on [GitHub](#).

The modular strategy-pattern architecture from Sprint 1 proved effective: adding `ActiveCaseCountFeature` as a new `PrefixFeature` required only minor changes to existing preprocessing or model code. The parametric configuration system and CLI entry point together mean that the full pipeline — from raw XES data to evaluation metrics — is now executable with a single command, satisfying REQ-IT-03 and REQ-DOC-01.

3.2 Difficulties

- **Time-series alignment edge cases.** Aligning time-series snapshots with case prefixes introduced subtle boundary conditions around timezone-aware versus timezone-naive timestamps, which caused comparison failures caught only during testing. This was resolved by normalising timestamp types in the preprocessing step before alignment.

- **Active-case-count performance.** The initial implementation computed active-case counts with a naive per-row loop over the full case list, which was prohibitively slow on the full BPI 2017 dataset. The lookup was optimised by pre-sorting timestamps and using `numpy.searchsorted`, achieving a significant speedup.
- **Outcome target coverage.** Some cases in the BPI 2017 log contain none of the three outcome-defining activities, making it impossible to assign a class label. A filtering step was added to `clean_event_log` to discard such cases before prefix generation, ensuring a clean training set for the classification task.
- **Configuration schema design.** Introducing a parametric run configuration (Issue #63) required agreeing on a Pydantic schema that could represent both classification and regression pipelines, all three model families, and the choice between log-only and hybrid feature sets. Several design iterations were needed before settling on the final structure.

3.3 Task Tracking

Sprint 2 was managed entirely within GitHub Issues, following the consolidation from Trello in Sprint 1. Parent issues represented deliverables; sub-issues tracked individual implementation tasks linked to pull requests. This structure proved effective: each deliverable’s completion status was immediately visible from the parent issue, and code review was consistently applied via pull requests before merging to `dev`.

4 Moving Forwards

4.1 Future Planning

Sprint 3 will focus on completing the experimental evaluation and producing the final written report. The planned work comprises the following tracks:

- **Feature importance analysis.** Feature importance scores will be computed and visualised for tree-based models, comparing the relative contribution of log-based and time-series features (REQ-RE-02). This will inform the critical discussion of whether and under which conditions the active-case-count signals add value.
- **Feature selection.** Based on the feature importance data, analysis and testing - new features will be proposed and selected in order to improve overall model performance. The currently implemented time-series features don’t improve model performance as expected, we plan to further improve on them via feature selection.
- **Rigorous comparative evaluation.** We will run the full experiment suite—baseline versus extended models, across both classification and regression tasks—and report performance broken down by prefix length (REQ-ME-04, REQ-RE-01). Statistical or qualitative significance assessments will be included as required by REQ-DOC-04.

- **Final report.** The 10–15 page final report (REQ-DOC-03, REQ-DOC-04) will cover data preprocessing and feature engineering choices, model specification and evaluation protocol, and a critical discussion of the added value of time-series features, including potential reasons for success or failure, computational overhead, and generalisability.

4.2 Role Allocation

Table 2. Role assignments for Sprints 1–3.

Role	Sprint 1	Sprint 2	Sprint 3
Project Coordinator / Scrum Master	Michał	Konstantinos	Roman
Technical Lead / Architecture	Roman	Michał	Konstantinos
Data & Preprocessing Lead	Roman, Konstantinos	Konstantinos	Roman
Modelling & Experimentation Lead	Michał	Roman	Michał
Evaluation & Visualization Lead	Konstantinos	Michał	Konstantinos
Quality & Tooling (Tests, CI)	Konstantinos	Roman	Roman
Documentation & Reporting	Michał, Roman, Konstantinos (shared)		

References

1. van Dongen, B.F.: BPI Challenge 2017. Eindhoven University of Technology (2017). <https://research.tue.nl/en/datasets/bpi-challenge-2017>